

INTRODUCTION

Basic Goal of Control

Apply inputs to drive the state of a system towards a reference (e.g. the origin)

Value Function Paradigm

Use a value function & dynamic programming to design a policy that maps states to inputs

Obstruction

Value function is unknown and must be estimated from a *finite number of noisy measurements*

LINEAR QUADRATIC REGULATOR

$$\begin{aligned} & \underset{\pi \in \Pi}{\text{minimize}} && \lim_{T \rightarrow \infty} \frac{1}{T} \mathbb{E}_{x_0, w_t} \sum_{t=0}^T \begin{bmatrix} x_t \\ u_t \end{bmatrix}^\top \begin{bmatrix} Q_{xx} & Q_{xu} \\ Q_{ux} & Q_{uu} \end{bmatrix} \begin{bmatrix} x_t \\ u_t \end{bmatrix} \\ & \text{subject to} && x_{t+1} = Ax_t + Bu_t + w_t \end{aligned}$$

x_t is the system state

u_t is the control input

w_t is i.i.d. process noise with zero mean and covariance W

A, B are the system matrices

Q is the positive definite penalty matrix

Π is the space of measurable history-dependent policies

$\pi = \{\pi_t\}$ with $u_t = \pi_t(x_{0:t}, u_{0:t-1})$

Matrix Operators

$$\mathcal{H}(P) := Q + [A \ B]^\top P [A \ B]$$

$$\mathcal{R}(P) := -P + \mathcal{H}_{xx}(P) - \mathcal{H}_{xu}(P) \mathcal{H}_{uu}^{-1}(P) \mathcal{H}_{ux}(P)$$

$$\mathcal{K}(P) := -\mathcal{H}_{uu}^{-1}(P) \mathcal{H}_{ux}(P)$$

Directional Derivative of Riccati Operator

Evaluated at point X in direction P

$$\mathcal{R}'(X, P) = -P + (A + BK(X))^\top P (A + BK(X))$$

LQR solved by linear state feedback $u_t = Kx_t$ with gain $K = \mathcal{K}(P)$ and P solves the *algebraic Riccati equation*

$$\mathcal{R}(P) = 0$$

LQR is a root-finding problem!

NEWTON METHODS

Goal: Find a root x of the vector equation $f(x) = 0$

Derive Newton methods from the Newton-Leibniz integral

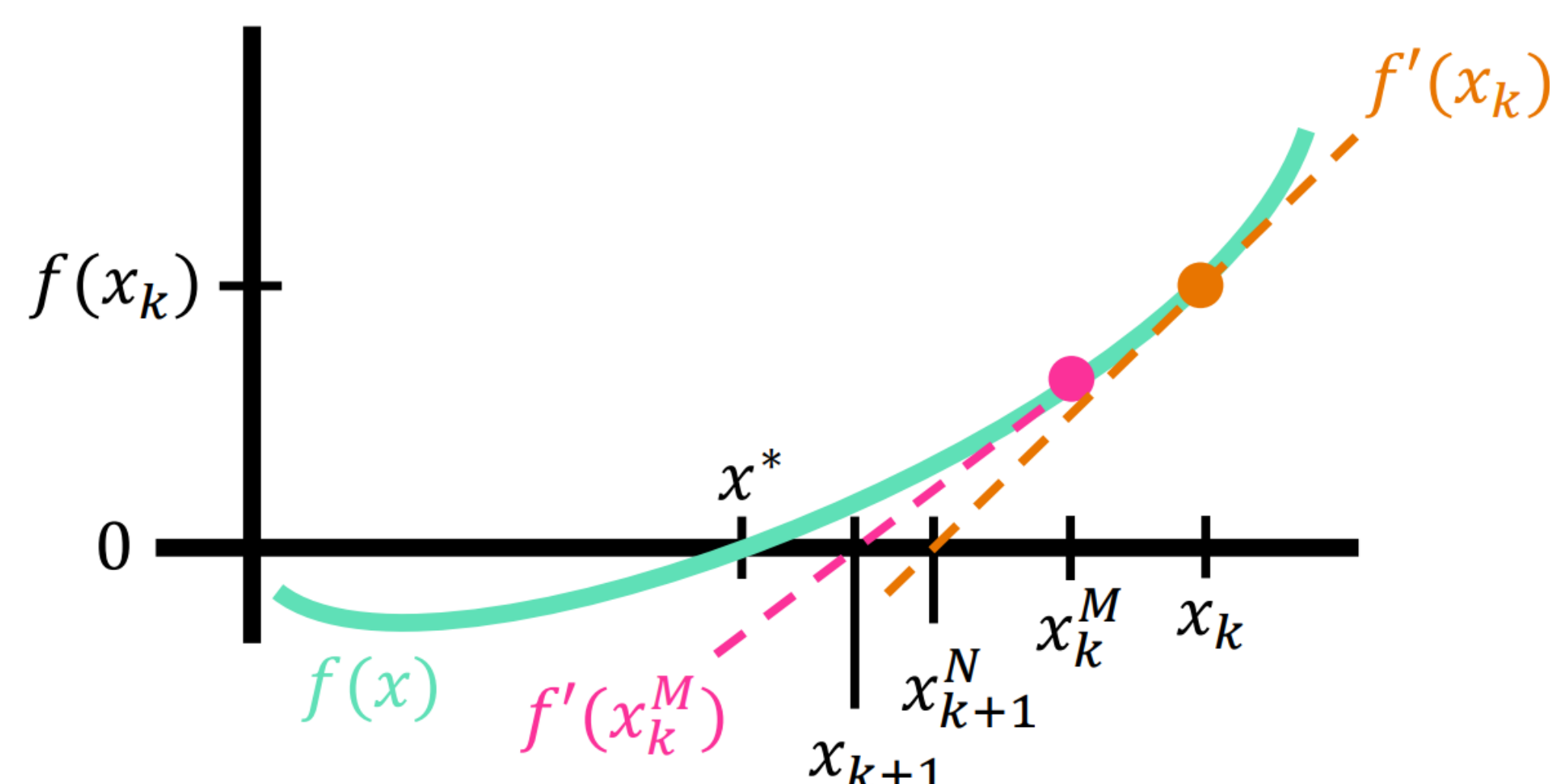
$$0 = f(x) = \int_{x_k}^x f'(t) dt$$

Left rectangular integration $\rightarrow$ Newton method

- Converges *quadratically*

Midpoint rectangular integration $\rightarrow$ Midpoint method

- Converges *cubically*
- Incurs only $\sim 2X$ computations vs Newton



- | | | |
|----------|---|---|
| Newton | { | <ol style="list-style-type: none"> Initial guess x_k Find <i>tangent hyperplane</i> $f'(x_k)$ Find zero crossing of <i>hyperplane</i> $f'(x_k)$ and set as x_{k+1}^N |
| | | |
| Midpoint | { | <ol style="list-style-type: none"> Find midpoint of x_k and x_{k+1}^N and set as x_k^M Find <i>tangent hyperplane</i> $f'(x_k^M)$ Find zero crossing of <i>hyperplane</i> $f'(x_k^M)$ and set as x_{k+1} |

POLICY ITERATION: CONNECTING OPTIMAL CONTROL & ROOT FINDING

General stochastic optimal control:

Policy iteration $\Leftrightarrow$ Newton method on Bellman equation [1]

Linear quadratic regulator: Kleinman-Hewer algorithm $\Leftrightarrow$ Newton method on algebraic Riccati equation [2]

Begs the questions

- What is the *analogue of the midpoint method for LQR?*
- Can we execute midpoint updates using *data-driven value function approximations*?

MIDPOINT POLICY ITERATION

Model Known

Compute value functions by solving Lyapunov equations (DLYAP)

Algorithm 1 Exact midpoint policy iteration (MPI)

while $k < N$ **do**

$$K_k = \mathcal{K}(P_k)$$

$$P_{k+1}^N = \text{DLYAP}(F^N, S^N)$$

$$M_k = \frac{1}{2}(P_k + P_{k+1}^N)$$

$$L_k = \mathcal{K}(M_k)$$

$$P_{k+1} = \text{DLYAP}(F^M, S^M)$$

Newton policy improvement

Collect data

Newton policy evaluation

Midpoint value function

Midpoint policy improvement

Collect data

Midpoint policy evaluation

Shift value function

Model Unknown

Estimate value functions using least-squares temporal differencing for Q-functions (LSTDQ) [3]

Algorithm 2 Approximate midpoint policy iteration (AMPI)*

while $k < N$ **do**

$$\hat{K}_k = -\hat{H}_{uu,k}^{-1} \hat{H}_{ux,k}$$

$$\mathcal{D}^N = \text{ROLLOUT}(\hat{K}_k, \ell, \mathcal{X}_0, \{\mathcal{U}_t\}_{t=0}^\ell)$$

$$\hat{H}_{k+1}^N = \text{LSTDQ}(\mathcal{D}^N, \hat{K}_k, Q)$$

$$\hat{H}_k^M = \frac{1}{2}(\hat{H}_k + \hat{H}_{k+1}^N)$$

$$\hat{L}_k = -\hat{H}_{uu,k}^{M-1} \hat{H}_{ux,k}^M$$

$$\mathcal{D}^M = \text{ROLLOUT}(\hat{L}_k, \ell, \mathcal{X}_0, \{\mathcal{U}_t\}_{t=0}^\ell)$$

$$\hat{H}_{k+1}^O = \text{LSTDQ}(\mathcal{D}^M, \hat{L}_k, \hat{Q}^M)$$

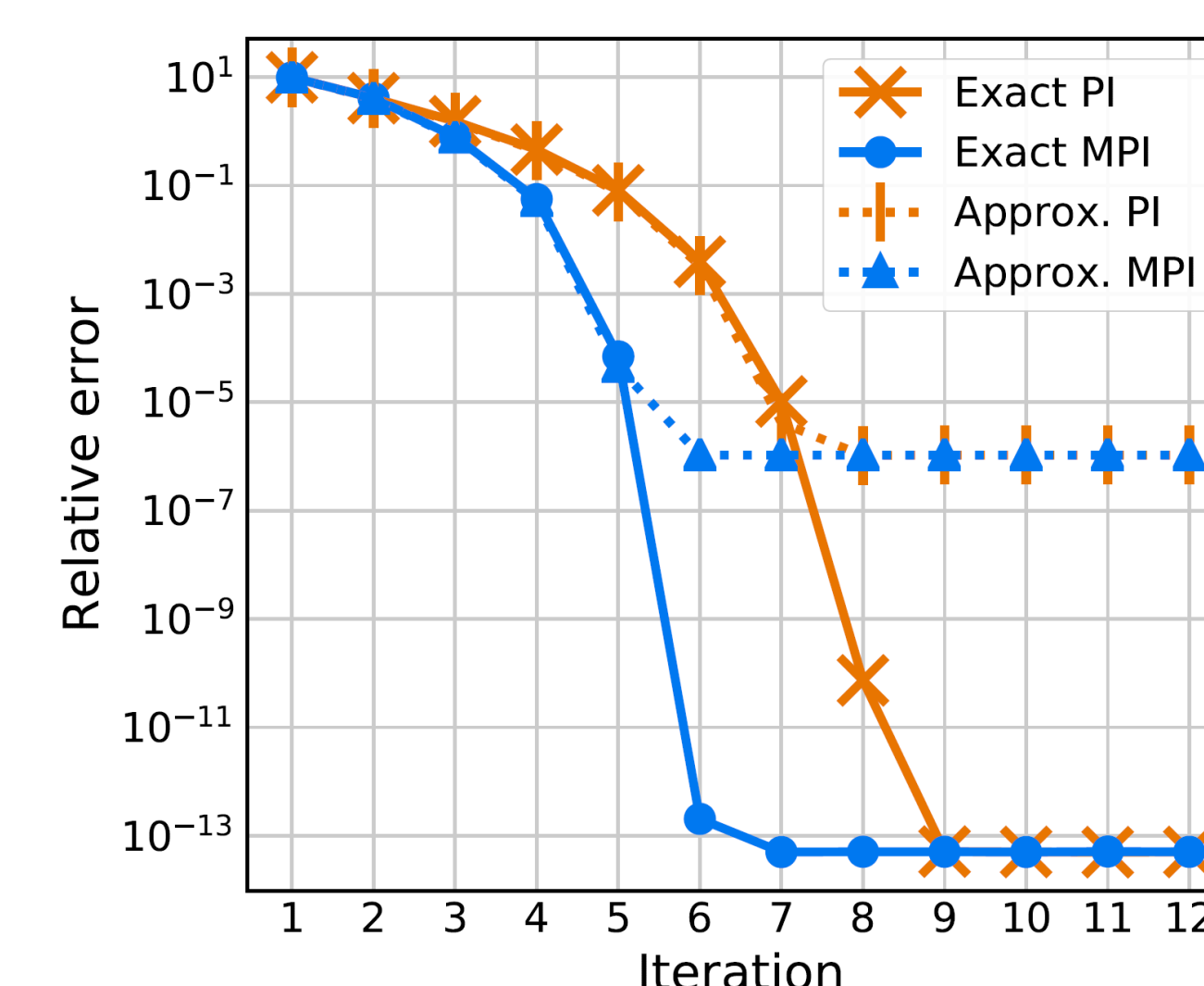
$$\hat{H}_{k+1} = \hat{H}_{k+1}^O + (Q - \hat{Q}^M)$$

*This is the on-policy version. The off-policy version uses a single dataset $\mathcal{D}$ collected once at the start.

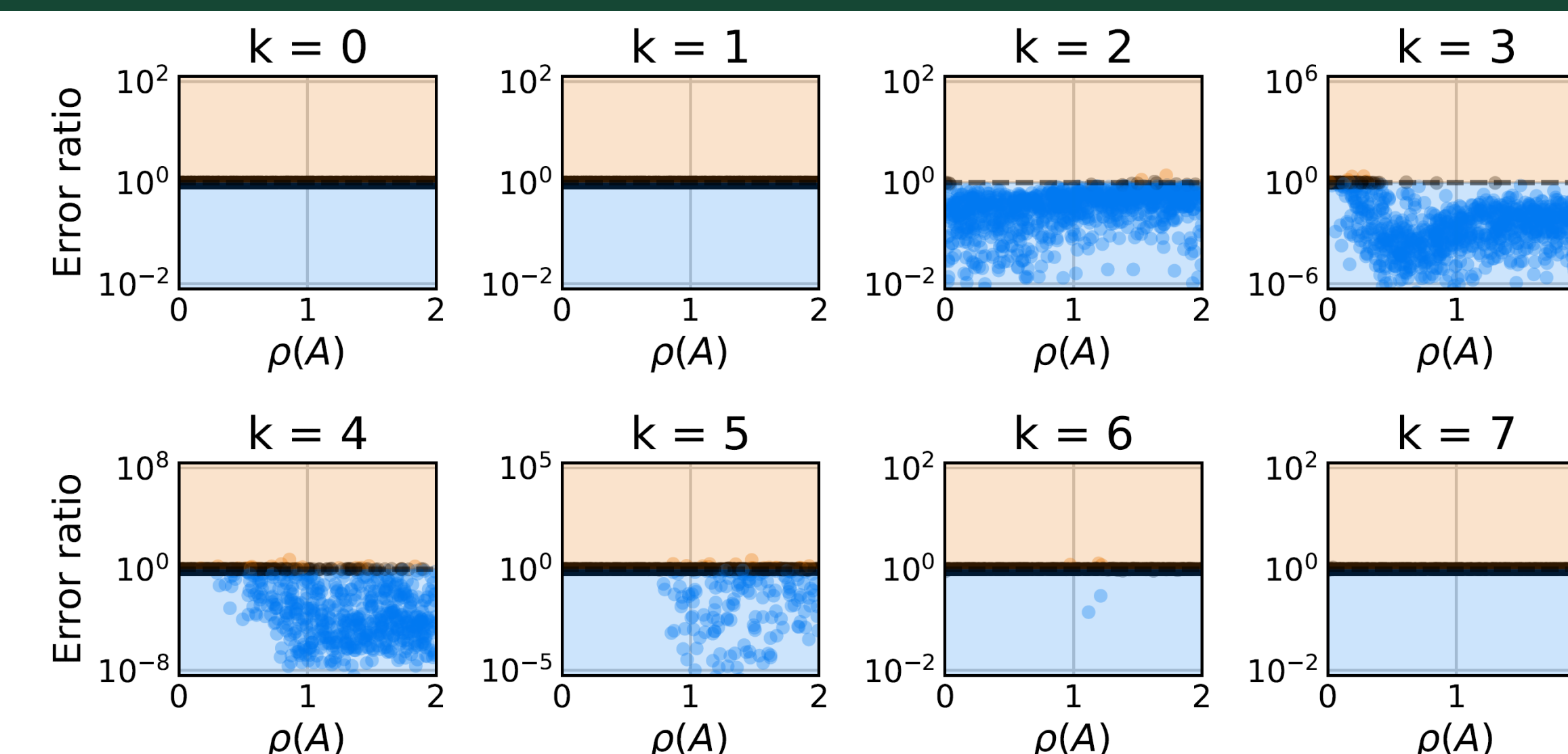
Local Cubic Convergence

If $\|K_0 - K^*\|$ small enough, then $\|K_{k+1} - K^*\| \leq \mathcal{O}(\|K_k - K^*\|^3)$ for all k .

NUMERICAL EXPERIMENTS



Relative error $\|P_k - P^*\|/\|P^*\|$ vs iteration count k using (A)PI and (A)MPI on an inertial mass control problem.



Ratio of relative errors $\|P_k - P^*\|/\|P^*\|$ using AMPI divided by that using API vs open-loop spectral radius on randomized control problems. Each subplot shows a single iteration count k .

CONCLUSIONS

We proposed a variant of policy iteration based on the midpoint Newton method with an enhanced convergence rate

Numerical experiments verify faster convergence, useful in the online setting where data is more expensive than computations

Future Work

- Global, non-asymptotic convergence guarantees.
- Extensions to multi-point and high-order Newton methods.
- Generalization to nonlinear systems.

ACKNOWLEDGEMENTS



This work is supported by the United States Air Force Office of Scientific Research under award number FA2386-19-1-4073 and by the National Science Foundation under award number ECCS-2047040.

TEASER	https://youtu.be/kllpEmXqBxc	PAPER	https://arxiv.org/abs/2011.14212
CODE	https://github.com/TSummersLab/midpoint-policy-iteration	CONTACT INFO	Benjamin Gravel Iman Shames Tyler Summers (PI)

[1] Martin L. Puterman and Shelby L. Brumelle. "On the convergence of policy iteration in stationary dynamic programming." Mathematics of Operations Research, 4(1):60–69, 1979.
[2] David Kleinman. "On an iterative technique for Riccati equation computations." IEEE Transactions on Automatic Control, 13(1):114–115, 1968.
[3] Michail G. Lagoudakis and Ronald Parr. "Least-squares policy iteration." Journal of Machine Learning Research, 4:1107–1149, December 2003.